Assignment

Question 1: Your friend wants to start a career in software, but thinks only people from big cities or top colleges can get jobs in IT companies.

Based on what you have learned about the software industry and career paths, what would you tell your friend? Explain what kinds of roles are available and what skills matter most to get started.

Answer:

It is a myth that only specific individuals can start a career in software, although being from big cities or top colleges can help with good exposure & networking, neither is required to have a successful career in IT. In today's modern IT industry, having a good skillset is much more important than coming from certain traditional backgrounds.

There are multiple entry-level roles to start your career from:

- Frontend Developer: Focuses on what users see and interact with on websites or applications using HTML, CSS, and JavaScript.
- Backend Developer: Handles the behind-the-scenes logic, servers, and databases that power applications using languages like Python, Java, or Node.js.
- Full-stack/MERN-stack developer: Handles both frontend and backend development.
- Quality Assurance (QA) Engineer / Tester: Tests software to find bugs, ensure high quality, and verify that features work correctly before release.

- Data Analyst: Collects, processes, and analyzes data to help businesses make informed decisions using tools like SQL and Excel.
- DevOps: Providing users with our product by deploying it. Uses CI|CD pipelining to send our final products/updates across the user platform.

Core Skills That Matter Most:

- Problem-Solving and Logic: The core of coding is breaking down complex problems into smaller, manageable steps.
- Proficiency in One Language: Mastery of one core language (such as Python or JavaScript) is better than a surface-level understanding of five.
- Version Control (Git/GitHub): Knowing how to track code changes and collaborate with others is essential for any modern development team.
- Continuous Learning: The industry evolves rapidly, so the ability to self-learn new tools, frameworks, and documentation is critical.
- Communication: Software development is a team sport; you must be able to explain your code, ask clear questions, and document your work.

Question 2: A small shop owner in your town wants to create a mobile app so that customers can order products online. He asks you — "How will this app actually be built? Who all will work on it?"

Explain to him the different roles involved in building software and how they work together as a team.

Answer:

There will be several people in multiple roles required for the completion of this app:

Product Manager – for understanding the goals of the shop owner and customer needs, then conveying that to the rest of the team.

Designer(UI/UX) – to design a visual outlook for the app.

Full-stack Developer – to code the various infrastructures, handle databases, user accounts, CSS, and secure the app.

QA Engineer – testing of the app's functionality by finding bugs before shipping the app to users.

DevOps Engineer – deploys using CI/CD pipeline

During the making of the product, we go through several steps/phases:

1. Planning: The Product Manager meets with the shop owner to map out features like a product catalog and shopping cart.
2. Designing: The UI/UX Designer turns those features into visual blueprints and clickable mockups for approval.
3. Development: Frontend and Backend Developers use those designs to write the actual code, connecting the visual screens to the database.
4. Testing: The QA Engineer tests the order process to ensure payments work securely and nothing crashes.
5. Deployment: The team launches the app on app stores.
6. Maintenance: The team, after launching the app on app stores, continues working together to release updates or fixing bugs.

Question 3: Your cousin is learning to code and wrote a program in Python. She clicked "Run" and got an error saying "SyntaxError". She is confused and asks you — "I wrote everything correctly, why is the computer not understanding me?"

Explain to her why computers cannot directly understand what we write, and what happens between writing code and the computer actually running it.

Answer:

The reason why Computers Cannot Directly Understand Our Code is because of a Language Barrier – Computers only speak "machine code," which is a series of binary numbers (0s and 1s) representing electrical signals. As we know, Python is a human-readable high-level language. Computers cannot execute it without a translation process. Computers lack human intuition and cannot guess intent; they require perfect adherence to structural rules. These so-called rules in a programming language are called syntax, which needs to be followed for us to make a computer do something.

What Happens Between Writing Code and Running It:
The Translator (Interpreter): Python uses a special program called an interpreter to translate code line-by-line into machine instructions.
The Syntax Check (Parsing): Before executing code, the interpreter reads it to verify that it follows all formal language rules.
The Syntax Error: If a rule is broken (like a missing colon or unmatched parenthesis), the translation halts immediately.
Execution: If the structure is perfect, the interpreter converts the code into binary instructions that the computer's CPU executes.

Question 4: You are asked to write step-by-step instructions (algorithm) for this problem: "A customer is buying items from an online store. If the total bill is more than ₹500, give a 10% discount. Otherwise, no discount."

Write the solution first in plain English steps, and then convert it into simple pseudocode.

Answer:

First, we need to check if the sum total of the items is more than 500 or not. For this, make a variable and store the value 500 in it, then use this variable to compare the total sum of the items. Only when the bill exceeds the 500 mark, provide a 10% discount.

Pseudocode:-

```
Assign the value 500 to a variable
If totalBill > minAmt:
    totalBill -= totalBill/10
else:
    return totalBill
```

Question 5: Your younger brother opens his phone browser and types "www.flipkart.com". Within 2 seconds, the Flipkart homepage appears on his screen. He asks you — "How did this happen so fast? Where did this page come from?"

Explain the complete journey — from the moment he types the website name to the moment the page shows up on his screen. Use simple terms like DNS, server, browser, HTTP/HTTPS in your explanation.

Answer:

The Journey of a Website Request:

1. Typing the URL: You type "flipkart.com" into your mobile browser.
2. The Address Book Lookup (DNS): The browser doesn't know where the website lives, so it asks the DNS (Domain Name System). The DNS acts like a phone contact list, translating the text name into a numeric IP address.
3. Sending the Request (HTTP/HTTPS): Armed with the IP address, your browser packages a request. It sends this securely across the internet using HTTPS protocols.
4. The Destination (Server): The request arrives at Flipkart's central computer, called a server, which stores all the website files, images, and code.
5. Sending the Data Back: The server processes the request and sends the website data back to your phone (IP address) using HTTPS.
6. Displaying the Page: Your browser receives the raw code files, translates them into text and images, and displays the homepage on your screen.

Question 6: A startup founder tells you — "I want to build a food delivery app like Zomato. I don't want to buy expensive servers and computers. Is there any other way?"

Explain to him what cloud computing is, why it is useful for startups, and give a simple example of how cloud services work.

Answer:

Cloud computing is renting data storage, processing power, and servers over the internet instead of buying and managing physical hardware yourself. Large tech companies (like Amazon AWS, Google Cloud, or Microsoft Azure) own massive data centers and let you use their infrastructure remotely.
Its Usefulness for Startups:

- Low Initial Cost: You do not need to buy expensive servers or hire data center maintenance teams upfront.
- Pay-as-You-Go: You only pay for the exact amount of computing power and storage space your application consumes.
- Easy Scaling: If your app goes from 10 users to 10,000 users overnight, you can instantly scale up your capacity with a few clicks.
- Faster Launch: You can launch a global infrastructure instantly, allowing you to focus purely on building your product.

A simple example for this can be domestic electricity:
Think of cloud computing like using electricity in a house. Instead of building your own power plant and buying massive generators just to turn on a light bulb, you plug your appliances into the wall outlet. The power company manages the infrastructure, and you simply receive a monthly bill based exactly on how much electricity you consumed.

Question 7: What is the difference between a compiler and an interpreter? Explain in your own words with a simple real-life comparison.

Answer:

Compiler: Translates the entire source code program into machine language all at once before running it. It generates an independent executable file (like an .exe). If there is an error anywhere in the code, the compilation fails, and no part of the program runs.

Example: Think of a compiler as translating a whole textbook from English to Spanish. A translator sits down, reads the entire book, rewrites it completely in Spanish, and hands you the finished Spanish book to read whenever you want.

Interpreter: Translates and executes the source code line-by-line in real time. It does not generate a separate executable file. It runs the program smoothly until it hits an error, at which point it halts immediately on that specific line.

Example: Think of an interpreter like a live tour guide translating for a tourist in real time. As the tourist speaks a sentence, the guide translates it immediately on the spot, line by line, as the conversation happens.

Question 8: What is the Software Development Life Cycle (SDLC)? Name all its phases and write one line about what happens in each phase.

Answer:

The Software Development Life Cycle (SDLC) is a structured, step-by-step process used by development teams to design, build, test, and maintain high-quality software efficiently.
The Phases of SDLC
Planning: Gathering business goals, defining user needs, and mapping out project timelines and costs.

Design: Creating visual blueprints, user interfaces, system architectures, and database layouts for the product.
Development: Writing the actual programming code using chosen languages to build the software features.
Testing: Checking the software for bugs, defects, and performance issues to ensure it functions perfectly.
Deployment: Launching the finalized, tested application into the real world or app store for customers to use.
Maintenance: Fixing new bugs, releasing software updates, and adding new features over time based on user feedback

Question 9: What is the difference between frontend development and backend development? Give one real example to show how both work together in a single application.

Answer:

Frontend Development: Refers to the "client-side" of an application. It involves everything users see, click, and interact with directly on their screens, built using tools like HTML, CSS, and JavaScript.

Backend Development: Refers to the "server-side" of an application. It involves the hidden logic, databases, authentication systems, and servers that power the application from behind the scenes.

Real Example: Logging Into an Account:

1. The Frontend Part: You open an app, see a sleek login form, and type your email and password into the text fields. When you tap the blue "Login" button, the frontend sends that text data across the internet.

2. The Backend Part: The backend receives your email and password, securely hashes it, checks it against the registered passwords stored in its database, and verifies your identity.
3. Working Together: Once verified, the backend sends a "success" signal back to the frontend, which immediately updates your screen to display your personalized user dashboard.

Question 10: What is the difference between HTTP and HTTPS? Why is HTTPS important when you are using a banking or payment website?

Answer:

HTTP (Hypertext Transfer Protocol): Sends data between your browser and the website server in plain text. Anyone intercepting the network traffic can read the information easily. This interception is known as a Man-in-the-Middle Attack, in which a third party can gain access to our request/data just by altering a router's code.

HTTPS (Hypertext Transfer Protocol Secure): Uses an encryption protocol (SSL/TLS) to scramble the data before sending it. Intercepted data looks like complete gibberish without the unique decryption key. This takes care of the Man-in-the-Middle Attack.

Why HTTPS is Critical for Banking and Payments:

- Data Encryption: It secures highly sensitive information like your credit card details, passwords, and bank account numbers from hackers.
- Identity Verification: It proves that you are connected to the bank's genuine, verified server rather than a fake replica website designed to steal your credentials.

- Data Integrity: It ensures that your payment transaction details cannot be tampered with or altered by an unauthorised third party while travelling across the internet.